Agent Diary User Guide (V1)

Introduction

Agent Diary is a local-first memory and inspection tool for one human and one agent.

Its purpose is to make agent memory easier to trust.

Most memory systems hide too much. You can often see a summary, but not the actual record underneath it. You may be able to ask the agent what it remembers, but not inspect what was stored, how that memory was derived, or what the agent actually did while working.

Agent Diary is meant to improve that.

In V1, it gives you one place to inspect:

- the raw conversation or diary entry
- the agent's supporting memory artifacts
- the agent's work between visible messages

The most important design rule is simple:

the diary entry is the main record, and everything else is attached around it.

Who This Guide Is For

This guide is for someone who wants to:

- run Agent Diary locally or on a headless host
- import real conversation data
- browse and inspect entries in the UI
- understand what the side panels mean
- use Agent Diary for a real week of evaluation

It is not mainly a developer architecture document. It is a practical user/operator guide for using the current V1 system.

What Agent Diary Is Trying To Solve

Agent Diary exists because there are really three different things people care about when working with an agent:

1. What was actually said
2. What the agent later inferred or summarized
3. What the agent actually did while working

Those are related, but they are not the same.

If they all get collapsed into one invisible “memory layer,” the system becomes hard to trust.

Agent Diary separates them.

The Core Model

Agent Diary has three main layers.

1. Raw entry

This is the primary truth layer.

Examples:

- imported Telegram conversation blocks
- imported OpenClaw session conversation entries
- manual diary notes

This is the thing shown in the main center reading view.

If you want to know what the source record actually is, this is where to start.

2. Agent work

This is the record of work the agent performed between visible messages.

Examples:

- commands run
- files changed
- tests executed
- imports performed
- decisions made
- blockers encountered
- handoff-like status updates

This matters because the visible conversation usually does not fully explain what happened during the work phase.

In V1, this appears in the right-side `Agent Work` panel attached to the entry you are viewing.

3. Derived artifacts

These are secondary interpretation layers.

Examples:

- conversation briefs
- compressed memory
- open-loop analysis
- overlays and stale warnings

These are useful, but they are not the primary record. They help you scan, recall, and inspect. They should not replace the raw entry.

How To Think About The Product

One useful mental model is:

- center panel = what happened in the diary record
- right panel = what the system knows around that record

That “around that record” context includes:

- interpretation
- annotation
- unresolved items
- work evidence

This is why the new `Agent Work` panel belongs on the right side rather than replacing the diary view itself.

What You Can Do In V1

V1 is already useful for:

- importing truthful conversation data
- browsing entries in a timeline
- searching memory
- opening an entry and reading the raw record
- inspecting conversation briefs and compressed memory
- inspecting open loops
- viewing attached agent work for that entry
- adding overlays and corrections

- refreshing derived layers after overlays or imports

V1 is not trying to be the final polished public version yet.

It is a serious working tool that should now be used for a week, then refined based on what feels confusing, missing, noisy, or genuinely helpful.

What V1 Is Not

V1 is not:

- a full replacement for every future memory workflow
- a polished consumer-grade release
- a chain-of-thought dump
- a fully automatic always-correct derived-analysis layer

In particular, the work-trace layer is about meaningful operational provenance, not about storing every tiny internal thought or every byte of terminal output.

Main UI Layout

The interface has three main areas.

Left column

This is the navigation and scope area.

It includes:

- search
- timeline
- recent imports
- provenance scope controls

This column answers:

- what corpus am I looking at
- what batch did this come from
- what entry do I want to open

Center panel

This is the `Diary Entry` view.

It shows:

- the raw entry body

- the primary diary text
- core metadata for the entry

This panel should be treated as the starting point for inspection.

If the raw entry and a derived interpretation disagree, the raw entry wins as the primary truth layer.

Right panel

This is the supporting interpretation and context area.

It includes:

- Open Loops
- Conversation Brief
- Compressed Memory
- Agent Work
- Overlays
- Refresh controls
- Secondary artifacts

This panel is where you inspect what the system inferred, what work was performed, and what extra context has accumulated around the entry.

Setup

Local single-machine setup

From the repo root:

```
cd /path/to/agent-diary
python3 -m venv .venv
source .venv/bin/activate
pip install -e .
```

Start the backend:

```
PYTHONPATH=src python3 -m agent_diary.cli.main serve --host
127.0.0.1 --port 8041
```

In a second terminal, start the UI:

```
cd ui
python3 -m http.server 5173
```

Then open:

```
http://127.0.0.1:5173
```

Headless host plus remote Mac setup

This is the Emily-style deployment pattern.

Backend and static UI run on the headless machine. The Mac connects over SSH tunnel.

Example:

```
ssh -L 5173:127.0.0.1:5173 -L 8041:127.0.0.1:8041 willard@<HOST>
```

Then open on the Mac:

`http://127.0.0.1:5173`

The UI will talk to the tunneled API at:

`http://127.0.0.1:8041`

If needed, confirm the `API Base` field in the header is set correctly.

What Happens During Normal Use

The normal flow is:

1. import real data
2. inspect the resulting import batch
3. browse entries in context
4. read the raw entry first
5. inspect right-side supporting layers when needed
6. add overlays if the record needs clarification
7. refresh derived layers if they are stale or missing

This keeps the product grounded in a truth-first workflow instead of treating derived layers as magic.

How To Use It

1. Import truthful conversation data

The normal operating pattern is:

1. import real conversation data
2. inspect the batch
3. browse entries
4. generate or refresh derived layers only after the truth layer is in place

For recurring truthful Telegram/OpenClaw import, use the established import commands documented in:

- `docs/guinea-pig-testing-quickstart.md`
- `docs/truthful-recurring-ingestion.md`

2. Inspect recent imports

Open `Recent Imports` in the left column.

Use it to:

- see the latest import batches
- confirm import counts
- apply scope to a specific import batch

Clicking an import batch should narrow the browse/search context automatically.

This is important because Agent Diary becomes much easier to reason about when you are working inside one bounded import context rather than the full corpus.

3. Read the diary entry first

When you open an entry:

- start with the center diary entry panel
- treat the raw entry as the main record
- use the right panel as supporting context

This is the intended V1 workflow.

It prevents the product from turning into a dashboard where the summary becomes more visible than the source.

4. Use the right panel for supporting context

Use the side panel to answer:

- what the agent inferred
- what is currently unresolved
- whether an overlay changed the interpretation
- what work the agent actually performed for this entry

Think of this as “attached context,” not a replacement narrative.

5. Use Agent Work when you want execution evidence

The `Agent Work` panel is for the work that happened between visible messages.

It may include:

- short work-event summaries
- event type
- actor / project / source surface
- touched paths
- related artifacts
- structured details

This panel is especially helpful when the conversation alone does not explain:

- why something changed
- what the agent actually did
- whether verification happened
- which files or tasks were involved

6. Use derived artifacts carefully

The right-side derived layers are useful, but they are still secondary.

Examples:

- a conversation brief helps with scanning
- compressed memory helps with fast recall
- open loops help identify unresolved items

These are support tools, not the same thing as the raw truth layer.

7. Add overlays when the record needs clarification

Use overlays for:

- annotations
- corrections
- clarifications

Overlays should not replace raw truth. They sit on top of it.

This is important because it preserves a clean audit trail:

- what the original entry was
- what was later clarified
- which derived artifacts may now be stale

8. Refresh derived layers when needed

Use the refresh controls when:

- new truthful entries were imported
- an overlay changed the meaning of an entry
- you want a fresh brief, memory layer, or open-loop analysis

Do not assume derived layers are always automatically current.

V1 is more trustworthy when refresh is explicit and inspectable.

Recommended Weekly Use Pattern

For the current test week, the simplest operator loop is:

1. import recent real data
2. open the latest import batch in the UI
3. browse entries in timeline order
4. read the raw entry first
5. inspect `Agent Work` when the action between messages matters
6. inspect derived layers when you want summaries or unresolved items
7. add overlays if the record needs explanation or correction
8. refresh derived layers only when needed
9. notice what feels awkward, unclear, missing, or especially useful

That is the whole point of the week: use it like a real tool and gather honest feedback.

Useful Commands

Start backend

```
PYTHONPATH=src python3 -m agent_diary.cli.main serve --host
127.0.0.1 --port 8041
```

Start UI

```
cd ui
python3 -m http.server 5173
```

List imports

```
PYTHONPATH=src python3 -m agent_diary.cli.main --json list-imports --limit 20
```

List entries

```
PYTHONPATH=src python3 -m agent_diary.cli.main --json list-entries --limit 20
```

Search memory

```
PYTHONPATH=src python3 -m agent_diary.cli.main --json search-memory --query "browser route" --limit 20
```

Fetch one entry

```
PYTHONPATH=src python3 -m agent_diary.cli.main --json fetch-raw-entry --entry-id <ENTRY_ID> --include-artifacts
```

Search work trace directly

```
PYTHONPATH=src python3 -m agent_diary.cli.main --json search-work-trace --query "timeline layout" --limit 20
```

Refresh conversation briefs

```
PYTHONPATH=src python3 -m agent_diary.cli.main --json produce-conversation-briefs --import-id <IMPORT_ID> --truthful-only --limit 200
```

Refresh compressed memory

```
PYTHONPATH=src python3 -m agent_diary.cli.main --json produce-compressed-memory --import-id <IMPORT_ID> --truthful-only --limit 200
```

Refresh open loops

```
PYTHONPATH=src python3 -m agent_diary.cli.main --json produce-open-loops --import-id <IMPORT_ID> --truthful-only --limit 200
```

Troubleshooting

UI opens but looks empty

Check:

- the backend is running on `127.0.0.1:8041`
- the UI server is running on `127.0.0.1:5173`
- the `API Base` field is correct
- your current scope is not accidentally too narrow

Timeline is too broad

Use:

- `Recent Imports`
- conversation scope
- import scope
- `truthful_only`

The tool is much easier to use when scoped.

Derived artifacts look wrong or outdated

Possible causes:

- new overlays were added after the artifact was generated
- you are looking at older derived state
- the entry scope is broader than intended

Try:

- checking the stale warning
- refreshing the relevant derived layer
- narrowing the scope

Agent Work panel is empty

That may mean:

- no linked work-trace events were recorded for that entry
- the entry came from a truth import without attached work-trace linkage
- the relevant workflow evidence exists elsewhere but is not yet linked to that entry

This is useful feedback during the test week and worth noticing.

Search feels noisy

Try narrowing by:

- import batch
- conversation scope
- truthful-only mode

Broad search is useful, but scoped search is usually easier to interpret.

Current V1 Limits

V1 is strong enough for real use, but it still has limits.

- It is still an operator-oriented tool, not a fully polished public app.
- Some concepts are still more technical than they should eventually be.
- Work-trace capture is meaningful, but not exhaustive.
- Derived layers still depend on explicit refresh in some situations.
- The best next improvements should come from actual use, not abstract guessing.

What To Pay Attention To This Week

While using Agent Diary this week, notice:

- whether the raw-first reading flow feels natural
- whether `Agent Work` answers the right questions
- whether the right panel is clear or cluttered
- whether import scope is easy to understand
- whether overlays and stale warnings behave sensibly
- whether the product helps you trust the record more
- whether any terminology still feels too technical

- whether the setup/use instructions are missing something important

That feedback should drive the next round of edits.

Final Notes

The goal of this V1 guide is not to pretend the product is finished.

The goal is to make the current system usable enough, understandable enough, and inspectable enough that a real week of use will generate high-quality feedback.

That feedback should be more valuable than another round of abstract planning.